

A Chameleon Advisor That Knows What Is Actually Free: Sorting the Artifact Set, Reading Live Blazar State, and Only Asking the Sites That Matter

Vivek Rai
Chameleon Fellowship
chi-edge-advisor / benchmark_v4

Abstract—This covers three pieces of work on the Chameleon advisor. First, we went through all 205 Trovi artifacts and found that only 95 of them actually contain Chameleon provisioning code, and only 90 are useful for recommending hardware. Second, we replaced the advisor’s broken availability code with one that talks to Blazar directly across all seven Chameleon sites, and it now knows how long a node stays free and when a busy one frees up. Third, we built a small database recording which node types live at which site, so the advisor only contacts the sites that matter, which is up to $10.8\times$ faster. We also explain why we dropped the tree structure for the artifact set in favour of flat records with tags: it is not just simpler, it retrieves better.

Index Terms—testbed provisioning, resource allocation, Blazar, retrieval, reproducibility

I. NOT BREAKING THE EXISTING BENCHMARK

Everything here risks disturbing the 556 benchmark answers we already collected, so we first pinned a snapshot to compare against. Rescoring all 556 cells, plus a checksum over the 63 input files, has to come out identical after any change. It does, and the gold gate still passes 50/50.

Pinning it turned up something worth knowing on its own: the snapshot already in the repository was out of date. It recorded 406 empty and 150 scored cells, but the answer files committed in that same commit actually rescore to 314 and 242. So there was no working baseline before this, only one that looked like one.

Nothing this work produces gets written into `benchmark_v4/grounding/`, because two scripts read everything in that folder and the benchmark’s stored checksum depends on what they find. Keeping the new files physically elsewhere is what guarantees that, rather than us remembering to be careful.

II. HOW MANY ARTIFACTS ARE ACTUALLY USABLE

We fetched all 205 artifacts (180 cloned normally, 20 partially because they were over a 200MB limit, 7 unreachable) and sorted them by whether they contain Chameleon provisioning calls.

This is the number that changes the plan. Just over half the catalog has no Chameleon provisioning code at all, usually because the GitHub link points at the paper’s own code while

TABLE I
WHAT IS IN THE 205 TROVI ARTIFACTS

Kind	n	What it means
no Chameleon code	108	nothing for the advisor to learn from
teaching material	49	tutorials, clean examples
research code	46	experiments, setup is incidental
excluded	2	already in v1, or explicitly dropped
usable	95	the real number, not 205

the Chameleon setup lives in the Trovi copy instead. Scoring the remaining 95 by how much concrete hardware detail we can actually pull out of them narrows it again: 39 in the top group and 51 in the second, so **90 artifacts** are worth loading. Any plan built around 205 is roughly twice as big as it needs to be.

There is a useful surprise going the other way. The new set contains **52 artifacts that call `add_node_reservation`**, the bare-metal way of reserving hardware. Our existing material contained that call zero times, except in lists of calls to avoid. So the new artifacts supply exactly the examples the advisor has never had.

We checked the negatives with a completely separate search over all 102 we could inspect, and found one genuine miss: a 9.4MB notebook skipped by a file size limit. Notebooks are large because of saved output images, not because of code, so the limit now applies to the code we pull out rather than the file.

III. WHY WE DROPPED THE TREE STRUCTURE

The advisor was originally built around a tree: pick the site, then the use case, then the artifact. We replaced it with flat records carrying tags, stored as JSON. This was not about simplicity. The tree finds the right artifact less often.

The tree is 17.8 points worse at finding the right artifact, and the reason is built into its shape. Each level is a gate you can fail permanently. If the use case is guessed wrong, the correct artifact is thrown out before the next level even runs, and that middle gate is only right 73.3% of the time. Flat

TABLE II
TREE VERSUS FLAT, HOW OFTEN THE RIGHT ARTIFACT IS FOUND (45 ITEMS)

Stage	Tree	Flat + tags
site	100.0 %	n/a
use case (top 2)	73.3 %	n/a
artifact (top 3)	68.9 %	86.7 %
retrieved text	68.9 %	86.7 %

scoring lets a weak match on one thing be outvoted by strong matches on everything else, instead of ending the search.

So the artifact set is now flat tagged records rather than a hierarchy: `catalog.jsonl`, an 18-column table in `manifest.tsv`, and `resource_catalog.json`. We keep two separate tag lists rather than one combined one, because the advisor is really doing a translation: 23 tags for what the user is trying to do, and 13 for what hardware that implies. The hardware tags come only from code we actually saw, never from prose, because those are the ones that become a real reservation.

IV. GETTING REAL AVAILABILITY DATA

The advisor’s availability code did not work. The public Chameleon reference API only serves static hardware descriptions, lists no CHI@Edge devices at all, and its deeper endpoints currently return server errors. The `python-chi` path failed while looking up its own site list, and could only ever see one site. We replaced both by talking to Blazar directly.

Three things we found shaped the design.

First, Blazar cannot filter for you. We tested four different query formats and every one returned all 132 hosts anyway. So the only way to ask for less is to contact fewer sites.

Second, a lease does not tell you which machines it holds. Reading the lease list alone finds nothing reserved and reports every machine as free, even at a site with an active lease. That mapping lives on a separate endpoint. This one fix moved the answer from “587 free out of 587” to Table III.

Third, there is a third state nobody was tracking. A node can be neither free nor reserved: it can be switched off for maintenance and simply not bookable. **111 of 655 nodes** were in that state and were being counted as free. Recommending one of those hands the user a reservation that cannot be submitted.

CHI@Edge needed separate handling: it answers on `/devices` rather than `/os-hosts`, and its hardware name sits in a different field (`device_type` is just “container” for every one of them). Every GPU worth wanting (A100, V100, H100) was fully booked when we measured, leaving only older cards free. That is exactly the situation the advisor exists to help with. We also record each node’s reservation windows now, so the advisor can say how long something stays free, and when a busy one frees up, instead of only whether it is free this second.

TABLE III
LIVE CHAMELEON HARDWARE, ALL SEVEN SITES

Site	Reads from	Free	Busy	Down	Total
CHI@Edge	devices	35	0	33	68
CHI@NCAR	os-hosts	35	8	1	44
CHI@NRP	os-hosts	0	0	4	4
CHI@NU	os-hosts	6	0	2	8
CHI@TACC	os-hosts	112	181	21	314
CHI@UC	os-hosts	18	93	21	132
KVM@TACC	os-hosts	2	54	29	85
Total		208	336	111	655

V. ONLY ASKING THE SITES THAT MATTER

Since we can only cut work by contacting fewer sites, we store which node types live where and refresh it periodically. The layout makes this pay off: of 41 node types, **37 exist at exactly one site** (2 at two sites, 2 at three), an average of 1.15 sites per type.

TABLE IV
HOW LONG IT TAKES TO CHECK AVAILABILITY

Request	Sites	Nodes	Time	Speedup
Everything (before)	7	655	34.9 s	1.0×
compute_skylake	3	57	18.9 s	1.8×
gpu_rtx_6000	1	37	7.5 s	4.7×
Pi camera job	1	46	3.2 s	10.8×

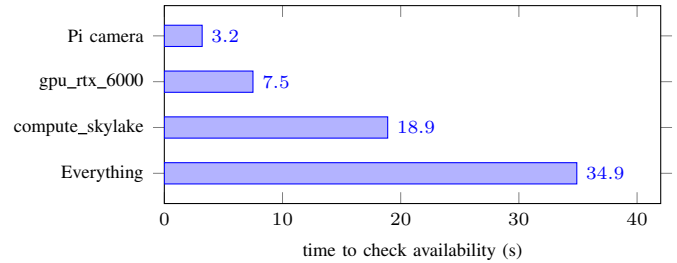


Fig. 1. Asking about specific hardware instead of everything. The fewer sites carry that hardware, the more you save.

We reordered the advisor to take advantage of this. Picking artifacts only needs the user’s question and the artifact list, no network at all, so it can run first and tell the availability step which hardware to ask about:

PICK ARTIFACTS → CHECK AVAILABILITY → DECIDE → CHECK

End to end, the Pi camera question now contacts one site and looks at 46 devices instead of 655, and still passes every check.

Two safeguards keep the speedup from causing wrong answers, since narrowing badly is worse than not narrowing at all. If we do not recognise the hardware being asked for, we go back to checking everything rather than guessing and returning nothing. And if the advisor ends up choosing hardware we did not fetch, it does one extra targeted lookup (3.5 s) instead of returning an empty list, which would make the checker think nothing is free and quietly fail a perfectly good answer.

VI. WRITING CODE FOR ALL SITES

The advisor’s recommendation now carries which site it is for and which style of reservation call that site needs, both copied from the stored database rather than guessed. The code generator picks one of three genuinely different styles.

TABLE V
THREE WAYS TO RESERVE, NOW HANDLED

Style	Types	Reservation call	Then runs
edge	7	add_device_reservation	Container
baremetal	31	add_node_reservation	Server
kvm	3	add_flavor_reservation	Server

Worth being clear about: the bare-metal template is written from the `python-chi` documentation, not from any working example we have, because those calls appear zero times in our current material. Both non-edge templates are labelled unverified in the generated code and are left out of the live check. They also cannot be reached yet: all five artifacts currently registered are CHI@Edge, so the advisor never picks non-edge hardware. The machinery is ready; giving it real examples is the next job.

VII. WHAT IS LEFT

- 1) **Load the 90 good artifacts** into the advisor. This blocks everything else: the multi-site code generation works but has no examples behind it, so a GPU training question still gets answered with a Raspberry Pi.
- 2) **Teach the advisor how to reserve, from the artifacts themselves**, using the 52 bare-metal examples we now have.
- 3) **Schedule the hardware database refresh**. It refreshes itself once a week if nobody does it, but nothing schedules it, so that refresh can hold up someone’s question for about 35 s.
- 4) **Check the 10 hardest artifacts by hand**, then the 25-artifact spread. Seven of those groups currently have only one member.
- 5) **Recover the extra hardware details** Blazar does not report for non-edge sites, and deal with the 4 artifacts stored only in Trovi, which is returning errors.

VIII. CONCLUSION

The artifact set is half the size it looked: 95 of 205 have Chameleon code, and 90 are worth using. The advisor now reads genuinely live data from all seven sites, including 111 machines switched off for maintenance that were being reported as free. Narrowing the question to specific hardware makes that data cheap enough to check on every single query, up to $10.8\times$ faster. The design lesson worth keeping is the tree result: the structure that felt obviously right was measurably worse, because every gate in it is a place you can be wrong and never recover.

Through all of it the frozen 556-cell benchmark came out identical, and the advisor’s tests stayed at 38 out of 38.